תבניות אלגוריתמיות לראיונות עבודה

שש עשרה התבניות שחוזרות ברוב שאלות הקוד. לכל תבנית: הסבר פשוט, מתי משתמשים, סיבוכיות עם הנימוק, ומחירים נסתרים. מונחים מסומנים בכוכבית (*) מוסברים במילון בסוף המסמך.

דף מרוכז – עמוד אחד לפני הראיון

שורה לכל תבנית: מה מזהה אותה בשאלה, מה המחיר, ומה הטעות שמפילה אנשים. הפירוט המלא בעמודים הבאים.

תבנית	רמז בשאלה	הרעיון בשורה	זמן	מקום	המלכודת
1. שני מצביעים	מערך ממוין, זוג או שלישייה שסכומם יעד, פלינדרום	שני אינדקסים מהקצוות, כל אחד זז לכיוון אחד	O(n)	O(1)	חייב מיון. מיון הורס אינדקסים מקוריים
2. חלון נע	"תת מערך רציף", "הארוך ביותר", "באורך k"	ימין מרחיב, שמאל מכווץ בלולאה עד שהתנאי תקין	O(n)	O(1)/O(k)	לא עובד עם מספרים שליליים בסכום. הכיווץ חייב להיות בלולאה
3. מהיר ואיטי	רשימה מקושרת, מעגל, איבר אמצעי	מצביע בקצב 1 ומצביע בקצב 2. נפגשים רק אם יש מעגל	O(n)	O(1)	לבדוק שהמהיר וגם הבא שלו אינם ריקים
4. חיפוש בינארי	ממוין, "המינימום שמקיים", לוג n נדרש	חוצים את הטווח בכל צעד. גם על "מרחב התשובות"	O(log n)	O(1)	לולאה אינסופית ממיד במקום מיד+1. גלישה בחישוב האמצע
5. סכום מצטבר	הרבה שאילתות "סכום מ-i עד j", "כמה תת מערכים סכומם k"	מערך עזר שכל תא בו הוא סכום כל מה שלפניו. שאילתה = חיסור	O(n) בניה, O(1) שאילתה	O(n)	הקלט חייב להיות קבוע. תא 0 שווה 0 חוסך תנאי קצה
6. מפת גיבוב	"ראינו כבר?", תדירויות, אנגרמות, מערך לא ממוין	מפתח ← ערך בגישה ישירה	O(1) ממוצע	O(n)	אין סדר. מפתח בלי דריסת השוואה וגיבוב נעלם. לא בטוח לחוטים
7. מיזוג קטעים	פגישות, חפיפות, זוגות התחלה וסוף	מיון לפי התחלה, ואז מיזוג עם האחרון בתוצאה	O(n log n)	O(n)	להחליט אם נגיעה בקצה היא חפיפה
8. מחסנית מונוטונית	"הבא הגדול יותר", טמפרטורות, היסטוגרמה, מי גשם	מחסנית של אינדקסים בסדר עולה או יורד. מוציאים כשמגיע גדול יותר	O(n)	O(n)	נראה ריבועי אך כל איבר נכנס ויוצא פעם אחת. לשמור אינדקסים
9. חיפוש לרוחב	"מינימום צעדים", "הקצר ביותר", לפי רמות, ללא משקלים	תור. מסמנים ביקור בכניסה לתור. לולאה פנימית לכל רמה	O(V+E)	O(V)	לא לגרף עם משקלים, שם דייקסטרה. סימון בהוצאה מהתור מנפח
10. חיפוש לעומק	איים, רכיבים, כל המסלולים, עצים	רקורסיה או מחסנית. עד הסוף לפני שמנסים ענף אחר	O(V+E)	O(H)	גלישת מחסנית במטריצות גדולות. לא מוצא מסלול קצר ביותר
11. נסיגה לאחור	"כל הצירופים", "כל התמורות", סודוקו, מלכות, n קטן	בחר, רד ברקורסיה, בטל. הוסף העתק לתוצאה	O(2ⁿ·n), O(n!·n)	O(n)	לשכוח את הביטול. להוסיף את הרשימה החיה במקום העתק
12. תכנות דינמי	"כמה דרכים", "המקסימום", תת בעיות שחוזרות	להגדיר מצב, נוסחת מעבר, בסיס. כל מצב פעם אחת	מצבים × עבודה	מצבים	הגדרת מצב לא מדויקת. לרוב אפשר לצמצם לשורה או שני משתנים
13. ערימה	"k הגדולים", חציון בזרם, מיזוג k רשימות	ערימת מינימום בגודל k. מוציאים את הקטן בכל חריגה	O(n log k)	O(k)	מינימום בשביל "הגדולים". אין מחיקה מהאמצע, אז מחיקה עצלה
14. איחוד וחיפוש	קבוצות, "מחוברים?", קשתות שמתווספות	מערך הורים. כיווץ מסלול ואיחוד לפי גודל	O(α(n)) ≈ O(1)	O(n)	אין ניתוק. לא נותן מסלול, רק כן או לא
15. מיון טופולוגי	תלויות, דרישות קדם, סדר ביצוע	דרגות כניסה. תור של אפסים. מורידים דרגה לשכנים	O(V+E)	O(V+E)	פלט קצר מ-V פירושו מעגל. לא לשכוח כיוון הקשת
16. חמדני	תזמון, קפיצות, בחירה מקומית ברורה	מיון לפי הקריטריון הנכון, ואז מעבר אחד	O(n log n)	O(1)	לא תמיד נכון. דוגמה קטנה ששוברת = תכנות דינמי

סולם הסיבוכיות

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

כמאה מיליון פעולות בשנייה. 10^6 = n:
ריבועי = שעות, n log n = חלקיק שנייה.
n עד 20: מעריכי בסדר. n עד 5000:
ריבועי בסדר.

שלוש שאלות לפני קוד

- הקלט ממוין? אם כן: שני מצביעים או חיפוש בינארי.
- המילה "רציף" מופיעה? חלון נע או סכום מצטבר.
- "כל" מול "המיטבי"? כל = נסיגה לאחור. מיטבי = תכנות דינמי או חמדני.

המשפטים לסיבוכיות

- "כל איבר נכנס ויוצא פעם אחת" – ליניארי מצטבר.
- "כל צעד חוצה את הטווח" – לוגריתמי.
- "מספר המצבים כפול העבודה למצב" – תכנות דינמי.
- "עלים בעץ הרקורסיה כפול עבודה בעלה" – נסיגה לאחור.

1. שני מצביעים `Two Pointers`

ההסבר הפשוט

דמיינו שורה של ילדים מסודרים לפי גובה. שני חברים עומדים, אחד בקצה הימני ואחד בקצה השמאלי, ומתקרבים זה לזה. בכל צעד הם מסתכלים על שני הילדים שלידם ומחליטים מי זז. כך עוברים על כל השורה פעם אחת במקום לבדוק כל זוג אפשרי.

מתי משתמשים

- מערך ממוין ומחפשים זוג שסכומו יעד.
- הסרת כפילויות במקום* ממערך ממוין.
- בדיקת פלינדרום, היפוך מערך.
- מיכל המים הגדול ביותר, שלישיות שסכומן אפס.
- מיזוג שני מערכים ממוינים.

הרמז המרכזי: הקלט ממוין, או שאפשר למיין אותו קודם.

סיבוכיות

זמן: `O(n)` . כל מצביע זז רק לכיוון אחד ולעולם לא חוזר. שני המצביעים יחד עושים לכל היותר n צעדים עד שהם נפגשים. אם צריך למיין קודם: `O(n log n)` .

מקום: `O(1)` . שומרים רק שני אינדקסים, בלי תלות בגודל הקלט.

מחירים נסתרים

- עובד רק כשיש סדר. על מערך לא ממוין צריך למיין קודם, וזה עולה יותר מהמעבר עצמו.
- מחזיר זוג אחד או ספירה. אם צריך את כל הזוגות, יכולים להיות `O(n²)` זוגות, ואז אין דרך לחסוך.
- מיון משנה את המערך המקורי. אם צריך את האינדקסים המקוריים, מפת גיבוב עדיפה.

```
int l = 0, r = arr.length - 1;
while (l < r) {
    int sum = arr[l] + arr[r];
    if (sum == target) return new int[]{l, r};
    if (sum < target) l++;   // need a bigger value: move the small side
    else r--;                  // need a smaller value: move the big side
}
```

2. חלון נע `Sliding Window`

ההסבר הפשוט

יש לכם מסגרת של תמונה ומחליקים אותה על שורה ארוכה של מספרים. כשהמסגרת זזה צעד ימינה, מספר אחד נכנס מצד אחד ומספר אחד יוצא מהצד השני. במקום לחשב הכל מחדש, רק מוסיפים את החדש ומחסירים את הישן.

מתי משתמשים

- חלון קבוע: הסכום המקסימלי של k איברים רצופים.
- חלון משתנה: התת מחרוזת הארוכה ביותר בלי תווים חוזרים.
- התת מערך הקצר ביותר שסכומו לפחות s.
- אנגרמות בתוך מחרוזת, עד k תווים שונים.

הרמז: המילים "רציף", "תת מערך", "תת מחרוזת" יחד עם "הארוך ביותר", "הקצר ביותר" או "באורך k".

סיבוכיות

זמן: `O(n)` . הקצה הימני מתקדם n פעמים. הקצה השמאלי מתקדם גם הוא לכל היותר n פעמים בסך הכל לאורך כל הריצה. סך הכל `2n` צעדים. זה ניתוח מצטבר*, לא לכל צעד בנפרד.

מקום: `O(1)` לחלון מספרי. `O(k)` כשצריך מפת תדירויות, כאשר k הוא גודל האלפבית (לאותיות אנגליות: 26, ולכן קבוע).

מחירים נסתרים

- דורש שהתנאי יהיה "מונוטוני"*: כשהחלון גדל, התנאי רק מתחזק או רק נחלש. סכום עם מספרים שליליים שובר את זה, ואז צריך סכום מצטבר עם מפת גיבוב.
- הטעות הנפוצה: לכווץ את החלון פעם אחת בלבד במקום בלולאה. הכיווץ חייב להיות בלולאה פנימית.
- קל להתבלבל בין "הקצה נכלל" ל"הקצה לא נכלל". להחליט מראש ולכתוב הערה.

```
int left = 0, best = 0;
Map<Character, Integer> count = new HashMap<>();
for (int right = 0; right < s.length(); right++) {
    count.merge(s.charAt(right), 1, Integer::sum);
    while (count.get(s.charAt(right)) > 1) {          // shrink until valid
        count.merge(s.charAt(left), -1, Integer::sum);
        left++;
    }
    best = Math.max(best, right - left + 1);
}
```

3. מצביע מהיר ואיטי Fast & Slow Pointers

ההסבר הפשוט

שני רצים במסלול. אחד רץ בקצב כפול מהשני. אם המסלול הוא קו ישר, המהיר מגיע לסוף ראשון וזהו. אם המסלול הוא מעגל, המהיר יקיף ויתפוס את האיטי מאחור. כך יודעים אם יש מעגל בלי לזכור איפה כבר היינו.

מתי משתמשים

- זיהוי מעגל ברשימה מקושרת ומציאת נקודת ההתחלה שלו.
- האיבר האמצעי של רשימה במעבר אחד.
- "מספר שמח", זיהוי מחזור בסדרה שכל איבר נגזר מהקודם.
- האיבר הכפול במערך של מספרים בטווח 1 עד n.

הרמז: רשימה מקושרת, או פונקציה שמובילה מכל ערך לערך הבא.

סיבוכיות

זמן: `O(n)` . בלי מעגל, המהיר מגיע לסוף אחרי `n/2` צעדים. עם מעגל באורך c, אחרי שהאיטי נכנס למעגל, המרחק ביניהם קטן באחד בכל צעד, ולכן הם נפגשים תוך לכל היותר c צעדים.

מקום: `O(1)` . שני מצביעים בלבד. זה היתרון הגדול על הפתרון הפשוט עם קבוצה של "כבר ביקרנו", שדורש `O(n)` מקום.

מחירים נסתרים

- הקוד קצר אך עדין. שכחה של בדיקת `fast.next != null` גורמת לחריגה.
- מציאת תחילת המעגל דורשת שלב שני שקשה להסביר בראיון. כדאי לתרגל את ההוכחה.
- קבוצה של "כבר ביקרנו" פשוטה יותר ומקובלת כשמותר להשתמש בזיכרון נוסף.

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true;   // cycle
}
return false;
```

4. חיפוש בינארי Binary Search

ההסבר הפשוט

משחק "נחש את המספר" בין 1 ל-1000. במקום לנחש 1, 2, 3, מנחשים 500. אם אומרים "יותר גדול", כל החצי התחתון נזרק. כל ניחוש מוחק חצי מהאפשרויות, ולכן מספיקים 10 ניחושים במקום 1000.

מתי משתמשים

- חיפוש ערך במערך ממוין.
- המופע הראשון או האחרון של ערך, נקודת הכניסה.
- מערך ממוין ומסובב, שיא במערך "הר".
- "חיפוש בינארי על התשובה": המהירות המינימלית לאכול את כל הבננות, הקיבולת המינימלית של הספינה. כשהתשובה היא מספר, והבדיקה "האם x מספיק" היא כן או לא באופן מונוטוני*.

סיבוכיות

זמן: `O(log n)`. כל איטרציה מקטינה את הטווח פי 2. אחרי k איטרציות נשארו `n / 2^k` איברים. הטווח מגיע לאחד כאשר `2^k = n`, כלומר $k = \log_2 n$. עבור מיליון איברים: כ-20 צעדים.

בחיפוש על התשובה: `O(n · log(range))`, כי כל בדיקה עוברת על הקלט ויש `log(range)` בדיקות.

מקום: `O(1)` בגרסה איטרטיבית. `O(log n)` בגרסה רקורסיבית בגלל מחסנית הקריאות*.

מחירים נסתרים

- הקלט חייב להיות ממוין. מיון עולה `O(n log n)`, ולכן משתלם רק כשיש הרבה חיפושים או כשהקלט כבר ממוין.
- הטעויות הנפוצות ביותר בראיונות: לולאה אינסופית מ-`left = mid` במקום `mid + 1`, וגלישה* ב-`(left + right) / 2`. הכתיב הבטוח: `left + (right - left) / 2`.
- על רשימה מקושרת אין גישה ישירה לאמצע, ולכן התבנית לא עובדת.

```
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;          // no overflow
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;
// "search on the answer": lo = min, hi = max; while (lo < hi) { mid; if (ok(mid)) hi = mid; else
```

5. סכום מצטבר `Prefix Sum`

ההסבר הפשוט

יש לכם קופת חיסכון ורושמים כל יום כמה כסף יש בה בסך הכל. כששואלים "כמה הכנסת בין יום 5 ליום 9?" לא סופרים מחדש. לוקחים את הרישום של יום 9 ומחסירים את הרישום של יום 4. חישוב אחד במקום חמישה.

מתי משתמשים

- הרבה שאילתות "סכום התת מערך מ-i עד j" על מערך שלא משתנה.
- מספר התת מערכים שסכומם k, בשילוב מפת גיבוב.
- סכום תת מטריצה במטריצה דו ממדית.
- "נקודת שיווי משקל": אינדקס שהסכום משמאלו שווה לסכום מימינו.

הרמז: "סכום של טווח", "כמה תת מערכים", מספרים שליליים שמונעים חלון נע.

סיבוכיות

בנייה: `O(n)` זמן ו- `O(n)` מקום. מעבר אחד, כל תא הוא התא הקודם ועוד האיבר הנוכחי.

שאילתה: `O(1)` . חיסור אחד: `prefix[j+1] − prefix[i]` .

סך הכל עבור q שאילתות: `O(n + q)` במקום `O(n · q)` בגישה הישירה.

מחירים נסתרים

- הקלט חייב להיות קבוע. אם המערך משתנה בין השאילתות, כל עדכון מחייב בנייה מחדש ב- `O(n)` . אז עוברים למבנה מתקדם יותר (עץ פנוויק או עץ קטעים) עם `O(log n)` לעדכון ולשאילתה.
- עולה זיכרון נוסף בגודל הקלט.
- הטריק של אינדקס 0 שווה 0 (מערך באורך `n + 1`) חוסך בדיקות קצה. בלעדיו הקוד מתמלא בתנאים.

```
long[] prefix = new long[n + 1];                // prefix[0] = 0
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + arr[i];
long rangeSum = prefix[j + 1] − prefix[i];      // sum of arr[i..j]

// count subarrays with sum == k (works with negatives)
Map<Long, Integer> seen = new HashMap<>(Map.of(0L, 1));
long run = 0; int count = 0;
for (int x : arr) { run += x; count += seen.getOrDefault(run − k, 0); seen.merge(run, 1, Integer::
```

6. מפת גיבוב `Hash Map / Hash Set`

ההסבר הפשוט

ארון עם אלף מגירות. כל חפץ מקבל מספר מגירה לפי השם שלו, בעזרת נוסחה קבועה. כשרוצים למצוא חפץ, מחשבים את המספר וניגשים ישר למגירה. לא צריך לחפש בכל הארון.

מתי משתמשים

- "האם כבר ראינו את זה?" בלי מיון.
- סכום זוג במערך לא ממוין: שומרים מה ראינו, ובודקים אם המשלים קיים.
- ספירת תדירויות: אנגרמות, התו הראשון שלא חוזר, הרוב.
- קיבוץ לפי מפתח: אנגרמות לקבוצות.
- מטמון* של תוצאות, זיכרון בתכנות דינמי.

בג'אווה: `HashMap` , `HashSet` . כשצריך סדר הכנסה: `LinkedHashMap` . כשצריך מיון לפי מפתח: `TreeMap` עם `O(log n)` .

סיבוכיות

הכנסה, חיפוש, מחיקה: `O(1)` בממוצע. פונקציית הגיבוב* מחשבת את המגירה ישירות. הממוצע מניח פיזור טוב של המפתחות.

במקרה הגרוע: `O(n)` כשכל המפתחות נופלים לאותה מגירה. בג'אווה 8 ומעלה, מגירה עמוסה הופכת לעץ, ולכן המקרה הגרוע הוא `O(log n)` .

מקום: `O(n)` , כניסה לכל מפתח.

מחירים נסתרים

- אין סדר. אם התשובה דורשת סדר, צריך מיון נוסף או מבנה אחר.
- עומס זיכרון גבוה: כל כניסה היא אובייקט, ומספרים נעטפים באובייקטים. למספרים בטווח קטן, מערך של מונים מהיר וזול בהרבה.
- מפתח עם דריסה שגויה של `equals` או `hashCode` שובר את המפה בשקט. מפתח שמשתנה אחרי ההכנסה "נעלם".
- לא בטוח לריבוי חוטים*. ריבוי חוטים דורש `ConcurrentHashMap` .

```
Map<Integer, Integer> seen = new HashMap<>();   // value -> index
for (int i = 0; i < nums.length; i++) {
    Integer j = seen.get(target - nums[i]);
    if (j != null) return new int[]{j, i};
    seen.put(nums[i], i);
}
```

7. מיזוג קטעים `Merge Intervals`

ההסבר הפשוט

יש לכם פתקים של פגישות: "מ-9 עד 11", "מ-10 עד 12", "מ-14 עד 15". קודם מסדרים את הפתקים לפי שעת ההתחלה. אחר כך עוברים לפי הסדר: אם הפגישה הבאה מתחילה לפני שהקודמת נגמרת, מדביקים את שני הפתקים לפתק אחד. אחרת, מתחילים פתק חדש.

מתי משתמשים

- מיזוג קטעים חופפים.
- הכנסת קטע חדש לרשימה ממוינת.
- "כמה חדרי ישיבות צריך": מספר החפיפות המקסימלי בו זמנית.
- זמנים פנויים משותפים לכמה עובדים.
- מספר הקטעים שצריך להסיר כדי שלא תהיה חפיפה.

הרמז: זוגות של התחלה וסוף, המילים "חפיפה", "פגישה", "קטע".

סיבוכיות

זמן: `O(n log n)`. המיון שולט. המעבר עצמו הוא `O(n)` כי כל קטע נבדק פעם אחת מול הקטע האחרון בתוצאה.

מקום: `O(n)` לרשימת התוצאה. המיון בג'אווה על מערך אובייקטים דורש עוד `O(n)` זמני.

לחדרי ישיבות: ממיינים התחלות וסופים בנפרד, או משתמשים בערימת מינימום של זמני סיום. שניהם `O(n log n)`.

מחירים נסתרים

- המיון הכרחי. בלי מיון, כל קטע צריך להיבדק מול כל האחרים: `O(n²)`.
- להחליט מראש אם נגיעה בקצה נחשבת חפיפה. "מ-9 עד 10" ו"מ-10 עד 11" – כן או לא? השאלה קובעת.
- בזרם של קטעים שמגיעים בזמן אמת המיון המוקדם לא אפשרי. אז עוברים ל-`TreeMap` עם `O(log n)` לכל הכנסה.

```
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
List<int[]> out = new ArrayList<>();
for (int[] cur : intervals) {
    if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0]) out.add(cur);
    else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], cur[1]);
}
```

8. מחסנית מונוטונית Monotonic Stack

ההסבר הפשוט

ילדים נכנסים לתור לפי הסדר, וכל ילד שואל "מי הילד הראשון אחריי שגבוה ממני?". שומרים ערימה של ילדים שעדיין מחכים לתשובה. כשמגיע ילד גבוה, הוא עונה לכל הנמוכים שמחכים בראש הערימה, והם יוצאים. הערימה תמיד מסודרת מהגבוה לנמוך, ולכן מספיק לבדוק את הראש.

מתי משתמשים

- "האיבר הבא הגדול יותר" או "הקטן יותר" לכל איבר.
- טמפרטורות יומיות: כמה ימים עד יום חם יותר.
- המלבן הגדול ביותר בהיסטוגרמה.
- לכידת מי גשם.
- סכום המינימומים של כל התת מערכים.

הרמז: לכל איבר צריך את "הראשון משמאל או מימין" שמקיים תנאי גודל.

סיבוכיות

זמן: $O(n)$. הלולאה הפנימית נראית מסוכנת, אבל כל איבר נכנס למחסנית פעם אחת ויוצא ממנה לכל היותר פעם אחת. סך כל ההוצאות לאורך כל הריצה חסום ב-n. זה ניתוח מצטבר*: לצעד בודד יכול להיות יקר, אבל הממוצע הוא קבוע.

מקום: $O(n)$ למחסנית במקרה הגרוע (מערך ממוין שכולו נכנס).

מחירים נסתרים

- קשה לזהות בפעם הראשונה. הפתרון הישיר עם שתי לולאות הוא $O(n^2)$ ומפתה.
- עדינות בשוויון: "גדול או שווה" מול "גדול ממש" משנה את התשובה ואת התנאי בלולאה.
- לשמור אינדקסים ולא ערכים. האינדקס נותן גם את המרחק וגם את הערך.

```
int[] next = new int[n]; Arrays.fill(next, -1);
Deque<Integer> stack = new ArrayDeque<>();      // indexes, values decreasing
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) next[stack.pop()] = i;
    stack.push(i);
}
```

9. חיפוש לרוחב BFS — Breadth-First Search

ההסבר הפשוט

זורקים אבן לבריכה. הגלים מתפשטים במעגלים: קודם כל מה שבמרחק צעד אחד, אחר כך כל מה שבמרחק שני צעדים, וכן הלאה. הפעם הראשונה שגל נוגע במטרה, זה בהכרח המסלול הקצר ביותר, כי כל מה שקרוב יותר כבר נבדק.

מתי משתמשים

- המסלול הקצר ביותר בגרף* שכל הקשתות בו שוות (ללא משקלים).
- מעבר על עץ לפי רמות, סדר זיגזג, הצד הימני של העץ.
- תפוזים רקובים, מרחק מהאפס הקרוב במטריצה. הרבה נקודות התחלה בו זמנית.
- סולם מילים: כמה החלפות מינימליות ממילה למילה.

הרמז: "המינימום צעדים", "הקצר ביותר", "לפי רמות".

סיבוכיות

זמן: `O(V + E)`. כל צומת נכנס לתור פעם אחת (בזכות סימון "ביקרנו"), וכל קשת נבדקת פעם אחת מכל צד.

במטריצה של `m × n`: `O(m · n)`, כי לכל תא יש לכל היותר 4 שכנים.

מקום: `O(V)` לתור ולקבוצת הביקורים. בעץ מלא, הרמה האחרונה מכילה כמחצית מהצמתים, ולכן התור יכול להיות גדול מאוד.

מחירים נסתרים

- לא מתאים לגרף עם משקלים שונים. שם צריך את דייקסטרה, שהוא חיפוש לרוחב עם ערימה במקום תור, ב-`O(E log V)`.
- לסמן "ביקרנו" בזמן ההכנסה לתור, לא בזמן ההוצאה. אחרת אותו צומת נכנס כמה פעמים והזמן מתנפח.
- צורך יותר זיכרון מחיפוש לעומק בגרפים רחבים.
- לספור רמות: לעבד את כל התור הנוכחי בלולאה פנימית לפני שעוברים לרמה הבאה.

```
Queue<int[]> q = new ArrayDeque<>(); boolean[][] seen = new boolean[m][n];
q.add(start); seen[start[0]][start[1]] = true;
int dist = 0;
while (!q.isEmpty()) {
    for (int size = q.size(); size > 0; size--) {          // one whole level
        int[] cur = q.poll();
        if (isTarget(cur)) return dist;
        for (int[] d : DIRS) { int r = cur[0] + d[0], c = cur[1] + d[1];
            if (inBounds(r, c) && !seen[r][c] && open(r, c)) { seen[r][c] = true; q.add(new int[]{
    }
    dist++;
}
```

10. חיפוש לעומק DFS — Depth-First Search

ההסבר הפשוט

מבוך. הולכים במסדרון עד הסוף. אם זה מבוי סתום, חוזרים צעד אחד ומנסים פנייה אחרת. מסמנים בגיר כל מקום שכבר היינו בו, כדי לא להסתובב במעגלים. בסוף עוברים על כל המבוך, אבל תמיד עד הסוף של מסלול אחד לפני שמנסים מסלול אחר.

מתי משתמשים

- מספר האיים במטריצה, גודל האי הגדול ביותר.
- רכיבי קשירות* בגרף, זיהוי מעגל.
- עומק עץ, האם עץ מאוזן, סכום מסלול, האב המשותף הנמוך ביותר.
- כל המסלולים מנקודה לנקודה.
- שכפול גרף, סריקת עץ בסדר תחילי, תוכי או סופי.

הרמז: "כל", "האם קיים", עצים כמעט תמיד, "רכיב", "אי".

סיבוכיות

זמן: $O(V + E)$. כל צומת מבוקר פעם אחת, כל קשת נבדקת פעם אחת מכל כיוון. בעץ: $O(n)$, כי $E = n - 1$.

מקום: $O(H)$ כאשר H הוא העומק המקסימלי, בגלל מחסנית הקריאות*. בעץ מאוזן $H = \log n$. בעץ מנוון (שרשרת) או במטריצה גדולה $H = n$, ואז ברקורסיה יש סכנת גלישת מחסנית*.

מחירים נסתרים

- רקורסיה קצרה ואלגנטית, אבל מחסנית הג'אווה מוגבלת (בדרך כלל כמה אלפי קריאות). מטריצה של 1000 על 1000 עם אי אחד גדול מפילה את התוכנית. הפתרון: מחסנית מפורשת.
- לא מוצא את המסלול הקצר ביותר. הוא מוצא מסלול כלשהו.
- סימון "ביקרנו" חובה בגרף עם מעגלים. בעץ אפשר לוותר.
- שינוי הקלט כסימון (למשל להפוך 1 ל-0 במטריצה) חוסך זיכרון, אבל הורס את הקלט. לציין זאת בראיון.

```
void dfs(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';                                  // mark visited (mutates input)
    dfs(g, r + 1, c); dfs(g, r - 1, c); dfs(g, r, c + 1); dfs(g, r, c - 1);
}
```

11. נסיגה לאחור Backtracking

ההסבר הפשוט

מרכיבים תלבושת מארון. לוקחים חולצה, אחר כך מכנסיים, אחר כך נעליים. אם הנעליים לא מתאימות, מורידים אותן ומנסים זוג אחר. אם שום נעליים לא מתאימות, מורידים גם את המכנסיים ומנסים מכנסיים אחרים. ככה עוברים על כל התלבושות האפשריות, אבל מפסיקים מוקדם כשמשהו לא מתאים.

מתי משתמשים

- כל התת קבוצות, כל התמורות, כל הצירופים שסכומם יעד.
- N מלכות, סודוקו, חלוקה לפלינדרומים.
- חיפוש מילה במטריצת אותיות.
- יצירת סוגריים תקינים.

הרמז: "כל האפשרויות", "כל הדרכים", "האם קיים סידור". התשובה עצמה גדולה מעריכית.

סיבוכיות

זמן: תלוי בגודל מרחב החיפוש. תת קבוצות: $O(2^n \cdot n)$, יש 2^n תת קבוצות וכל אחת מועתקת ב- $O(n)$. תמורות: $O(n! \cdot n)$. גיזום* מקטין בפועל אך לא משנה את החסם העליון.

הדרך לנמק: מספר העלים בעץ הרקורסיה כפול העבודה בכל עלה.

מקום: $O(n)$ לעומק הרקורסיה ולמסלול הנוכחי, בלי לספור את הפלט.

מחירים נסתרים

- מעריכי מטבעו. מתאים ל-n קטן (בדרך כלל עד 20). אם n גדול, המראיין מצפה לתכנות דינמי או לחמדני.
- הטעות הקלאסית: לשכוח את שלב ה"ביטול" אחרי הרקורסיה, או להוסיף את הרשימה עצמה לתוצאה במקום העתק שלה.
- כפילויות בקלט דורשות מיון ודילוג על ערכים חוזרים באותה רמה.

```
void backtrack(int start, List<Integer> path, List<List<Integer>> out) {
    out.add(new ArrayList<>(path));                  // copy, not the live list
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);                             // choose
        backtrack(i + 1, path, out);                   // explore
        path.remove(path.size() - 1);                  // un-choose
    }
}
```

12. תכנות דינמי Dynamic Programming

ההסבר הפשוט

עולים במדרגות, וכל פעם אפשר לעלות מדרגה אחת או שתיים. כמה דרכים יש להגיע למדרגה 10? במקום לספור מחדש, שמים על כל מדרגה פתק: "אליי אפשר להגיע ב-X דרכים". הפתק של מדרגה 10 הוא הפתק של 9 ועוד הפתק של 8. כל פתק נכתב פעם אחת, ואף חישוב לא חוזר על עצמו.

מתי משתמשים

- "כמה דרכים", "המקסימום", "המינימום", "האם אפשר", כשבעיה גדולה מורכבת מבעיות קטנות זהות שחוזרות.
- מדרגות, מטבעות, תיק גב, תת סדרה עולה הכי ארוכה, תת מחרוזת משותפת, מרחק עריכה.
- הסכום המקסימלי של תת מערך רציף (אלגוריתם קדאן).
- שני תנאים לזיהוי: תת בעיות חופפות* ותת מבנה אופטימלי*.

שתי גישות: מלמעלה למטה עם זיכרון (רקורסיה ומטמון), ומלמטה למעלה עם טבלה.

סיבוכיות

זמן: מספר המצבים כפול העבודה למצב. מדרגות: n מצבים, `O(1)` למצב, סך הכל `O(n)`. תיק גב: `n · W` מצבים, `O(n · W)`. מרחק עריכה: `O(m · n)`.

מקום: כמספר המצבים. לעיתים קרובות אפשר לצמצם: אם מצב תלוי רק בשורה הקודמת, שומרים שתי שורות במקום טבלה שלמה. מדרגות: שני משתנים במקום מערך, `O(1)`.

מלמעלה למטה מוסיף `O(depth)` למחסנית הקריאות.

מחירים נסתרים

- מלמעלה למטה: קל לכתוב מהרקורסיה הטבעית, מחשב רק מצבים נחוצים, אבל מסתכן בגלישת מחסנית ואיטי יותר בגלל קריאות.
- מלמטה למעלה: מהיר ובלי רקורסיה, מאפשר צמצום זיכרון, אבל דורש למצוא את סדר המילוי הנכון ומחשב את כל המצבים.
- קל לטעות בהגדרת המצב. להגיד בקול "מה בדיוק תא `dp[i]` מייצג" לפני שכותבים.
- ניסוח ההגדרה של המצב הוא 80 אחוז מהפתרון. אחריו, נוסחת המעבר ותנאי הבסיס.

```
// bottom-up, O(n) time, O(1) space
int a = 1, b = 1;                       // ways(0), ways(1)
for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
return b;

// top-down with memo
int ways(int n, int[] memo) {
    if (n <= 1) return 1;
    if (memo[n] != 0) return memo[n];
    return memo[n] = ways(n - 1, memo) + ways(n - 2, memo);
}
```

13. ערימה Heap / Priority Queue — Top-K

ההסבר הפשוט

ערימה היא קופסה שכל פעם ששואלים ״מי הכי קטן?״ היא עונה מיד, בלי לחפש. כשמכניסים או מוציאים, היא מסדרת את עצמה מחדש בכמה צעדים בודדים. לא מסודרת עד הסוף, אבל הקטן ביותר תמיד למעלה.

מתי משתמשים

- k האיברים הגדולים או השכיחים ביותר.
- k הנקודות הקרובות למקור.
- מיזוג k רשימות ממוינות.
- החציון בזרם נתונים: שתי ערימות, מקסימום ומינימום.
- תזמון משימות, חדרי ישיבות, דייקסטרה.

בג׳אווה: `PriorityQueue` היא ערימת מינימום. למקסימום: `Collections.reverseOrder()`.

סיבוכיות

הכנסה והוצאה: `O(log k)` כאשר k הוא גודל הערימה. הערימה היא עץ בינארי כמעט מלא, גובהו `log k`, והתיקון אחרי שינוי עובר לאורך מסלול אחד מהשורש לעלה.

הצצה לראש: `O(1)`.

k הגדולים מתוך n: מחזיקים ערימת מינימום בגודל k. כל איבר נכנס ומוציאים את הקטן. `O(n log k)` זמן, `O(k)` מקום. עדיף על מיון מלא `O(n log n)` כש-k קטן מ-n.

בניית ערימה מ-n איברים בבת אחת: `O(n)`, לא `O(n log n)`.

מחירים נסתרים

- ערימת מינימום עבור ״k הגדולים״: מפתיע וטועים בו. שומרים את ה-k הגדולים, וזורקים תמיד את הקטן שבהם.
- אין חיפוש יעיל של איבר שרירותי, ואין מחיקה יעילה מהאמצע: `O(n)`. כשצריך את זה, `TreeMap` או ״מחיקה עצלה״*.
- אלגוריתם ״בחירה מהירה״ נותן k הגדולים ב-`O(n)` ממוצע, אך במקום ולא בזרם.
- הסדר בתוך הערימה אינו ממוין. הדפסה שלה לא נותנת רשימה ממוינת.

```
PriorityQueue<Integer> heap = new PriorityQueue<>();     // min-heap of size k
for (int x : nums) {
    heap.offer(x);
    if (heap.size() > k) heap.poll();                          // drop the smallest
}
return heap.peek();                                            // k-th largest
```

14. איחוד וחיפוש Union-Find (Disjoint Set)

ההסבר הפשוט

בכיתה יש קבוצות של חברים. לכל קבוצה יש "נציג". כדי לדעת אם שני ילדים באותה קבוצה, שואלים כל אחד "מי הנציג שלך?" ומשווים. כששתי קבוצות מתאחדות, הנציג של אחת מצביע על הנציג של השנייה. ובכל פעם ששואלים, מקצרים את השרשרת כדי שבפעם הבאה התשובה תהיה מיידית.

מתי משתמשים

- מספר רכיבי הקשירות בגרף לא מכוון.
- זיהוי מעגל בגרף לא מכוון, "הקשת המיותרת".
- מספר המחוזות, חשבונות עם אותו בעלים.
- עץ פורש מינימלי (קרוסקל).
- שאילתות "האם מחוברים" שמגיעות בזרם, כשהגרף רק גדל.

הרמז: "קבוצות", "מחוברים", "איחוד", קשתות שמתווספות עם הזמן.

סיבוכיות

עם שתי האופטימיזציות (כיווץ מסלול ואיחוד לפי גודל או דרגה): כל פעולה היא $O(\alpha(n))$, פונקציית אקרמן ההפוכה*. לכל n מעשי הערך קטן מ-5, ולכן מתייחסים לזה כקבוע.

בלי אופטימיזציות: $O(n)$ לפעולה במקרה הגרוע (שרשרת).

בנייה: $O(n)$ זמן ומקום למערך ההורים.

סך הכל עבור E קשתות: $O(E \cdot \alpha(n))$, בפועל $O(E)$.

מחירים נסתרים

- תומך רק באיחוד. אין "ניתוק" יעיל. אם קשתות נמחקות, המבנה לא מתאים.
- לא נותן מסלול, רק "כן או לא". למסלול צריך חיפוש לרוחב או לעומק.
- חיפוש לעומק עושה את אותה עבודה על גרף סטטי באותו זמן, ופשוט יותר להסביר. איחוד וחיפוש מנצח כשהקשתות מגיעות בהדרגה.
- ניתוח הסיבוכיות קשה להוכחה. בראיון מספיק להגיד "כמעט קבוע בזכות כיווץ מסלול".

```
int[] parent, size;
int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }   // path compressi
boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false;                      // already connected (cycle)
    if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
    parent[rb] = ra; size[ra] += size[rb];           // union by size
    return true;
}
```

15. מיון טופולוגי `Topological Sort (Kahn's Algorithm)`

ההסבר הפשוט

מתלבשים בבוקר: גרביים לפני נעליים, חולצה לפני מעיל. יש כללים של "לפני", אבל לא לכל זוג. מתחילים מכל דבר שאין לו שום דרישה, לובשים אותו, ומוחקים אותו מהדרישות של האחרים. כל פריט שנשאר בלי דרישות נכנס לתור. אם בסוף נשארו פריטים שלא נלבשו, יש מעגל: "נעליים לפני גרביים וגרביים לפני נעליים".

מתי משתמשים

- לוח קורסים עם דרישות קדם: האם אפשר לסיים, ובאיזה סדר.
- סדר בנייה של מודולים עם תלויות.
- מילון חייזרים: שחזור סדר האותיות ממילים ממוינות.
- זיהוי מעגל בגרף מכוון*.
- שורש מינימלי לעץ (קילוף עלים).

הרמז: "תלות", "לפני", "דרישת קדם", "סדר ביצוע". הגרף חייב להיות מכוון וללא מעגלים.

סיבוכיות

זמן: `O(V + E)`. חישוב דרגות הכניסה עובר על כל הקשתות פעם אחת. אחר כך כל צומת נכנס לתור ויוצא ממנו פעם אחת, וכל קשת יוצאת ממנו מורידה דרגה פעם אחת.

מקום: `O(V + E)` לרשימת השכנויות*, ועוד `O(V)` למערך הדרגות ולתור.

בדיקת מעגל: אם מספר הצמתים בפלט קטן מ-V, יש מעגל. בלי עלות נוספת.

מחירים נסתרים

- עובד רק על גרף מכוון ללא מעגלים*. מעגל אחד, ואין סדר תקין. זה לא חיסרון, זו התשובה.
- יש בדרך כלל כמה סדרים תקינים. אם השאלה דורשת "הסדר הלקסיקוגרפי הקטן ביותר", מחליפים את התור בערימה: `O(V log V + E)`.
- גרסה רקורסיבית עם חיפוש לעומק קיימת גם היא (סדר סופי הפוך), אך זיהוי המעגל בה דורש שלושה צבעים ומבלבל יותר.
- לרוב השאלה נותנת רשימת זוגות, ובניית הגרף היא חלק מהעבודה. לא לשכוח את כיוון הקשת.

```
int[] indeg = new int[n]; List<List<Integer>> adj = ...;
for (int[] e : edges) { adj.get(e[0]).add(e[1]); indeg[e[1]]++; }   // e[0] before e[1]
Queue<Integer> q = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indeg[i] == 0) q.add(i);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int u = q.poll(); order.add(u);
    for (int v : adj.get(u)) if (--indeg[v] == 0) q.add(v);
}
boolean hasCycle = order.size() < n;
```

16. חמדני Greedy

ההסבר הפשוט

נותנים עודף עם מטבעות של 10, 5 ו-1. תמיד לוקחים את המטבע הגדול ביותר שנכנס, ולא חושבים קדימה. עם המטבעות האלה זה תמיד נותן הכי מעט מטבעות. אבל עם מטבעות של 1, 3 ו-4, בשביל 6 החמדן ייתן 4 ועוד 1 ועוד 1, בעוד שהתשובה הנכונה היא 3 ועוד 3. חמדני עובד רק כשהבחירה הטובה עכשיו לא הורסת את העתיד.

מתי משתמשים

- תזמון: מקסימום פגישות בלי חפיפה, בוחרים תמיד את זו שמסתיימת ראשונה.
- משחק הקפיצות: האם אפשר להגיע לסוף, ומינימום קפיצות.
- תחנות דלק, חלוקת עוגיות, בלוני חצים.
- מניה: קנייה ומכירה, רווח מקסימלי.
- קידוד הופמן, דייקסטרה, קרוסקל, כולם חמדניים בבסיסם.

הרמז: "המקסימום או המינימום" כשיש כלל מיון או בחירה מקומית ברור.

סיבוכיות

בדרך כלל `O(n log n)` בגלל מיון מקדים, ואחריו מעבר `O(n)`. אם אין צורך במיון (קפיצות, מניה): `O(n)`.

מקום: `O(1)` מעבר למיון. זה היתרון הגדול על תכנות דינמי, שדורש טבלה.

הניתוח קל. הקושי הוא ההוכחה שהבחירה החמדנית נכונה. שיטת ההוכחה: "טיעון החלפה", מראים שכל פתרון אופטימלי אפשר להפוך לחמדני בלי להרע אותו.

מחירים נסתרים

- מהיר ופשוט, אבל לא תמיד נכון. בעיית המטבעות עם 1, 3 ו-4 היא הדוגמה הקלאסית. כשלא בטוחים, תכנות דינמי הוא הבחירה הבטוחה.
- בראיון, לומר את הבחירה החמדנית בקול ולתת דוגמה נגדית קטנה או נימוק קצר. פתרון חמדני בלי הצדקה נראה כמו ניחוש.
- כלל אצבע: אם דוגמה קטנה שוברת את החמדן, זו בעיית תכנות דינמי.

```
// max non-overlapping intervals: pick the one that ends first
Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
int count = 0, end = Integer.MIN_VALUE;
for (int[] it : intervals) if (it[0] >= end) { count++; end = it[1]; }

// jump game: furthest reachable index
int reach = 0;
for (int i = 0; i <= reach && i < nums.length; i++) reach = Math.max(reach, i + nums[i]);
return reach >= nums.length - 1;
```

סדר עבודה בראיון

1. לחזור על השאלה במילים שלכם ולשאול על קצוות: קלט ריק, כפילויות, מספרים שליליים, גודל הקלט.
2. להגיד את הפתרון הישיר ואת הסיבוכיות שלו. זה מוכיח הבנה ונותן נקודת ייחוס.
3. לזהות את התבנית לפי הרמזים בטבלה שבעמוד הראשון, ולהגיד למה היא מתאימה.
4. לכתוב קוד. לדבר תוך כדי. שמות ברורים, בדיקות קצה בהתחלה.
5. להריץ דוגמה קטנה ביד. אחת רגילה, אחת קצה.
6. לסכם: זמן, מקום, ומה היה אפשר לשפר עם עוד זמן.

מילון מונחים

סימון O גדול

דרך לתאר כמה מהר גדל זמן הריצה כשהקלט גדל, בלי להתעסק בקבועים. `O(n)` אומר: אם הקלט גדל פי 2, הזמן גדל בערך פי 2. `O(n²)` אומר: פי 4. `O(log n)` אומר: הזמן גדל בצעד אחד בלבד. הסדר מהמהיר לאיטי: `O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)` . כלל אצבע: מחשב עושה כמאה מיליון פעולות פשוטות בשנייה. אם n הוא מיליון, `O(n²)` הוא טריליון פעולות, כלומר שעות. `O(n log n)` הוא כעשרים מיליון, כלומר חלקיק שנייה.

ניתוח מצטבר (אמורטיזציה)

לפעמים צעד בודד יקר, אבל הצעדים היקרים נדירים. במקום לשאול "כמה עולה הצעד הגרוע" שואלים "כמה עולים כל הצעדים ביחד, חלקי מספר הצעדים". דוגמה: במחסנית מונוטונית, איבר אחד יכול לגרום להוצאה של אלף איברים, אבל כל איבר יוצא רק פעם אחת בחייו, ולכן סך כל ההוצאות הוא n. דוגמה נוספת: הגדלת מערך דינמי פי 2 כשהוא מתמלא. ההעתקה יקרה, אבל קורית רק אחת ל-n הכנסות.

מונוטוני

הולך רק בכיוון אחד: רק עולה או רק יורד, בלי לחזור. חלון נע עובד רק כשהתנאי מונוטוני: אם החלון "טוב", חלון קטן יותר בתוכו גם טוב. חיפוש בינארי על התשובה עובד רק כשהבדיקה מונוטונית: אם x מספיק, כל ערך גדול מ-x גם מספיק.

במקום

אלגוריתם שלא מקצה זיכרון נוסף בגודל הקלט. הוא משנה את הקלט עצמו ומשתמש בכמה משתנים בודדים. היתרון: מקום `O(1)` . המחיר: הקלט המקורי נהרס.

מחסנית הקריאות

כשמתודה קוראת למתודה, המחשב שומר "איפה הייתי" בערימת פתקים. כל קריאה רקורסיבית מוסיפה פתק. הפתק נזרק כשהמתודה מסתיימת. לכן רקורסיה בעומק n צורכת `O(n)` זיכרון גם אם לא הקצינו שום מערך. בג'אווה המחסנית מוגבלת, בדרך כלל בין 512 קילובייט למגה אחד, שזה כמה אלפי עד כמה עשרות אלפי קריאות.

גלישת מחסנית

שגיאת `StackOverflowError` . קורית כשהרקורסיה עמוקה מדי והמחסנית מתמלאה. הפתרון: להחליף רקורסיה בלולאה עם מחסנית מפורשת מסוג `ArrayDeque` , שנמצאת בערימת הזיכרון הרגילה ואינה מוגבלת כך.

גלישה מספרית

מספר שלם בג'אווה מוגבל לכ-2.1 מיליארד. חיבור שני מספרים גדולים חורג מהגבול, והתוצאה "מתגלגלת" למספר שלילי בלי שגיאה. לכן במקום `(lo + hi) / 2` כותבים `lo + (hi − lo) / 2` , ובסכומים גדולים משתמשים ב- `long` .

פונקציית גיבוב

נוסחה שמקבלת מפתח (מחרוזת, מספר, אובייקט) ומחזירה מספר. המספר קובע באיזו "מגירה" המפתח יושב. פונקציה טובה מפזרת מפתחות שונים למגירות שונות. שני מפתחות שונים באותה מגירה הם "התנגשות", והיא מאטה את המפה. בג'אווה זו המתודה `hashCode()` , וחייבים לדרוס אותה יחד עם `equals()` : שני אובייקטים שווים חייבים לתת אותו קוד גיבוב.

מטמון וזיכרון (ממואיזציה)

שמירת תוצאה של חישוב כדי לא לחשב אותה שוב. בתכנות דינמי מלמעלה למטה, לפני כל חישוב בודקים ״כבר חישבתי את זה?״ במפה או במערך. אם כן, מחזירים מיד. זה הופך רקורסיה מעריכית לליניארית.

תת בעיות חופפות

הרקורסיה הטבעית מגיעה לאותה תת בעיה שוב ושוב. חישוב פיבונאצ׳י של 50 מחשב את פיבונאצ׳י של 10 מיליוני פעמים. זה הסימן שתכנות דינמי יעזור. אם כל תת בעיה מופיעה פעם אחת (כמו במיון מיזוג), אין מה לשמור.

תת מבנה אופטימלי

הפתרון הטוב ביותר לבעיה הגדולה מורכב מפתרונות טובים ביותר לבעיות הקטנות. המסלול הקצר ביותר מ-א׳ ל-ג׳ דרך ב׳ מכיל את המסלול הקצר ביותר מ-א׳ ל-ב׳. בלי התכונה הזאת אי אפשר לבנות את התשובה מלמטה.

גיזום

בנסיגה לאחור, לעצור ענף ברגע שברור שהוא לא יוביל לפתרון. למשל, אם הסכום הנוכחי כבר עבר את היעד וכל המספרים חיוביים, אין טעם להמשיך. הגיזום לא משנה את החסם העליון, אבל בפועל מקצר את הריצה מאוד.

גרף, צומת, קשת

גרף הוא אוסף של נקודות (צמתים, מסומנים V) וקווים ביניהן (קשתות, מסומנות E). מפת ערים וכבישים היא גרף. גרף מכוון: לכל קשת יש כיוון, כמו רחוב חד סטרי. גרף לא מכוון: אפשר לעבור לשני הכיוונים. מטריצה היא גרף שבו כל תא הוא צומת וכל תא מחובר לארבעת שכניו.

גרף מכוון ללא מעגלים

גרף מכוון שאי אפשר בו לצאת מצומת ולחזור אליו. עץ תלויות תקין הוא כזה. רק על גרף כזה יש מיון טופולוגי.

רכיב קשירות

קבוצת צמתים שכולם מגיעים זה לזה. ״אי״ במטריצה הוא רכיב קשירות. גרף יכול להכיל כמה רכיבים שאין ביניהם שום קשת.

רשימת שכנויות

הדרך הנפוצה לייצג גרף: לכל צומת, רשימה של שכניו. בג׳אווה: `List<List<Integer>>` או `Map<Integer, List<Integer>>`. תופסת `O(V + E)` מקום. החלופה, מטריצת שכנויות, תופסת `O(V²)` ומשתלמת רק בגרפים צפופים.

פונקציית אקרמן ההפוכה

מסומנת `α(n)`. פונקציה שגדלה לאט יותר מכל פונקציה שנפגוש בפועל. עבור n בגודל מספר האטומים ביקום, הערך שלה עדיין קטן מ-5. כשאומרים שאיחוד וחיפוש הוא `O(α(n))`, מתכוונים ״קבוע לכל דבר ועניין״.

מחיקה עצלה

ערימה לא יודעת למחוק מהאמצע. במקום למחוק, מסמנים את האיבר כ״מת״ במפה נפרדת. כשהאיבר עולה לראש הערימה, בודקים אם הוא מת, ואם כן זורקים אותו וממשיכים. כך שומרים על `O(log n)` לכל פעולה.

בטיחות בריבוי חוטים

כשכמה חוטים (תהליכונים) משנים מבנה נתונים בו זמנית, המבנה יכול להישבר בשקט. `HashMap` ו-`ArrayList` אינם בטוחים. בראיון, השאלה ״מה קורה אם עשרה חוטים קוראים לזה בו זמנית״ היא מסורתית. התשובה: מבנה מקבילי כמו `ConcurrentHashMap`, או נעילה עם `synchronized`, או להפוך את המבנה לבלתי ניתן לשינוי.

יציבות מיון

מיון יציב שומר את הסדר המקורי של איברים שווים. `Arrays.sort` על אובייקטים בג׳אווה יציב (מיון מיזוג משופר). על מספרים פרימיטיביים אינו יציב (מיון מהיר כפול), אבל לפרימיטיבים זה לא משנה. חשוב כשממיינים לפי מפתח אחד ורוצים לשמור מיון קודם לפי מפתח אחר.